# Pixels, Frames & Transformers

Vision-Language Models as Annotators · Module 10

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

# A Research Assistant That Can See

A **vision-language model** (VLM) is a language model that can look at images. You can:

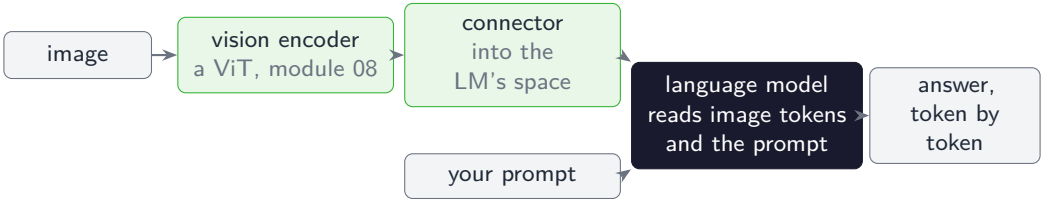
- ask what a picture shows
- have it fill in a coding sheet
- have it describe a scene for a text pipeline

Small open VLMs now run on a free Colab GPU.

**Treat it like a new research assistant**
Give it your codebook, check its work against yours, study its mistakes, and trust it only where it has earned that trust.

The same workflow as Day 4 of Bags, Vectors & Transformers: prompt, validate, distill.

# How a VLM Is Built




Qwen3-VL merges each $2 \times 2$ block of patches into one token, so a 768-pixel image becomes a few hundred tokens. The **language model** is where the knowledge, the instruction following and the hallucinations come from.

# Open Models That Fit a Free GPU

| model | license | note |
|---|---|---|
| **Qwen3-VL** 2B/4B | Apache 2.0 | our default, runs on a T4 |
| Qwen3.5 2B/4B | Apache 2.0 | newer, but needs a GPU with bfloat16 (not a T4) |
| Molmo2 4B | Apache 2.0 | its training data is open too |

On a T4 (no bfloat16) the notebook runs the 2B model in float32: about 9 GB, slower, but float16 can overflow and garble answers.

**Why open weights matter**
A versioned open model gives the same answers next year. A commercial API can change the model behind it without notice, and you can't inspect what it was trained on.

This landscape changes every few months; check before you start a project.

# Your Codebook as a Prompt

```
You are coding photos for a study of
protest.
Code only what is visible in the photo.
- protest:  people engaged in a protest:
...
  Code 0 for ...a festival, a market ...
- crowd:  roughly ten or more people ...
- police:  ...Security guards don't count.
- signs:  ...Street names don't count.
Answer with JSON only, in exactly this
format:
{"protest":  0, "crowd":  0, "police":  0,
"signs":  0}
```

- Definitions *and* rules for hard cases, as in module 04
- **JSON** output: easy to read into a table, and obvious when the format breaks
- **Greedy decoding**: same image, same prompt, same answer
- Keep every raw answer, and count the ones that failed

# Validate It Like a Human Coder

Per variable, against your codes:

- agreement and **kappa** (rare variables can have high agreement and low kappa)
- the **share coded yes**, by you and by the model: a model that says yes too often biases every prevalence you estimate
- errors per group or context, if you compare groups (module 07)

**Look at the disagreements**
Some are model errors, some are cases the codebook doesn't settle, and some are your own slips. Human codes are the benchmark, not the truth.

Answers that failed are often the hard images. Never drop them silently.

# Hallucination

A language model writes the most plausible continuation, and a plausible description of a protest includes police and signs, whether or not the photo shows them (Rohrbach et al., 2018).

**A quick test**: on images you coded *without* police, compare

- a neutral question: "Is there a police officer? Answer yes or no."
- a leading one: "Describe what the police officers are doing."

**Leading questions invite invention.** Small models often describe officers who aren't there, even after answering "no" to the neutral question.

Ask neutral questions with an explicit "0 = not visible", and treat open descriptions as claims to check.

## Sensitivity Checks

**Prompt wording**
A shorter prompt with the same variables in another order. Report how often the two agree, as you would for two human coders.

**Image resolution**
768 versus 336 pixels: about a fifth of the image tokens. Details suffer first (police at the edge, text on a placard); the overall scene suffers least.

Also worth checking when relevant: the order of images in a multi-image prompt, the model size, and a newer version of the same model.

# Scaling Up: Two Routes

**Describe, then analyze**

The VLM writes two sentences per image; text methods do the rest (here TF-IDF and a logistic regression).

+ transparent: you can read what the model mentions
− slow, and whatever isn't put into words is lost

**Distillation**

The VLM codes a few hundred images; a linear probe on CLIP embeddings learns from those codes and then labels the whole collection in minutes.

+ fast at any scale
− inherits the VLM's errors and adds its own

Either way, the final classifier is validated on human codes like anything else.

## What to Report

1. **The model, exactly**: name, size, weights version, library version, precision, decoding settings
2. **The full prompt**, and how and how often answers broke the format
3. **Image handling**: resolution, cropping, anything removed
4. **Validation** per variable on a random human-coded sample: kappa or F1, and the share coded yes by humans and by the model; per group if you compare groups
5. **Sensitivity** to wording and resolution, and whether the findings hold
6. **Hallucination checks** for variables the model could invent
7. **If you distilled**: the VLM-labeled training set and the final classifier's validation

# In the Notebook

- Loading Qwen3-VL-2B and asking it about images
- The module 04 codebook as a JSON prompt, validated variable by variable
- Catching hallucinations with leading questions
- Sensitivity to prompt wording and image resolution
- Describe, then analyze with a text classifier
- Distilling VLM labels into a CLIP probe

part3_transformers/10_vlms_as_annotators/notebooks/
10_vlms_as_annotators_exercises.ipynb · 6 exercises · GPU needed